

# CSC 317

Tutorial: Ray Tracing

# Assignment 2 Reminders

- Assignment 2 Due Date: September 19 before 11:59 pm
- Questions and Assistance
  - “Issues” page on github for Assignment 2
  - Email the TAs: [csc317tas@cs.toronto.edu](mailto:csc317tas@cs.toronto.edu)
- Office Hour via [Zoom](#): September 19, 11:00 am - 1:00 pm

# Assignment 3

- Assignment 2 Due Date: September 26 before 11:59 pm
- Questions and Assistance
  - “Issues” page on github for Assignment 3
  - Email the TAs: [csc317tas@cs.toronto.edu](mailto:csc317tas@cs.toronto.edu)
- Office Hour via [Zoom](#): September 26, 9:00 am - 11:00 am

# Prerequisites - all from previous 2 assignments

src/Plane.cpp

src/Sphere.cpp

src/Triangle.cpp

src/TriangleSoup.cpp

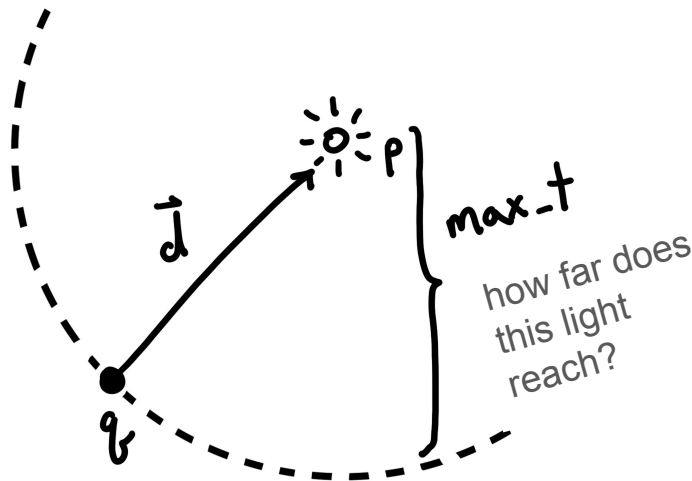
src/first\_hit.cpp

src/viewing\_ray.cpp

src/write\_ppm.cpp

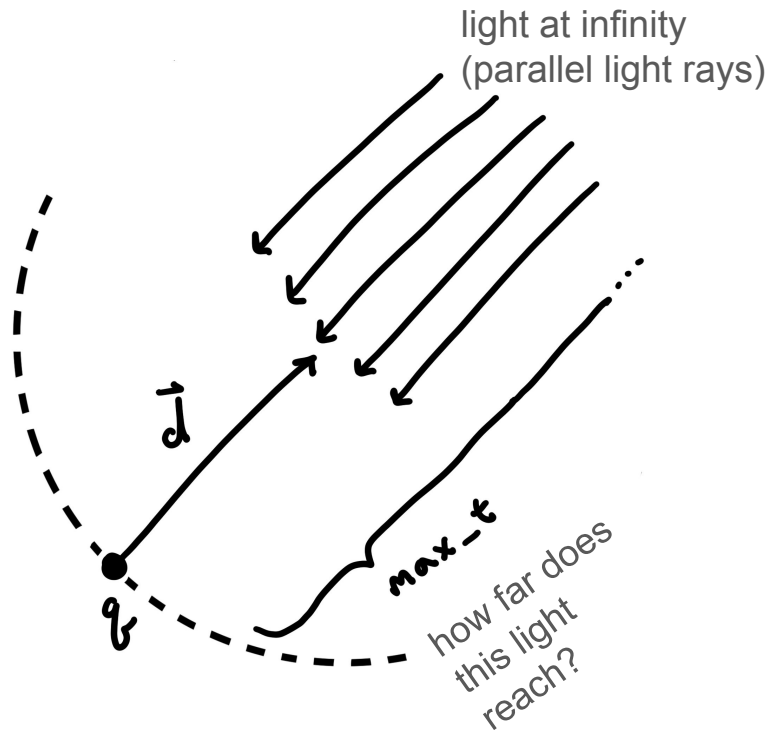
# PointLight.cpp direction

- Input:
  - $q$ : query point in space
- Output:
  - $d$ : vector direction from query point  $q$  towards this light's location  $p$ ; unnormalized
  - $max\_t$ : maximum (parametric) extent of this point light (i.e. how far this light can reach); parametric relative to  $d$
- Check .h file for what members are defined



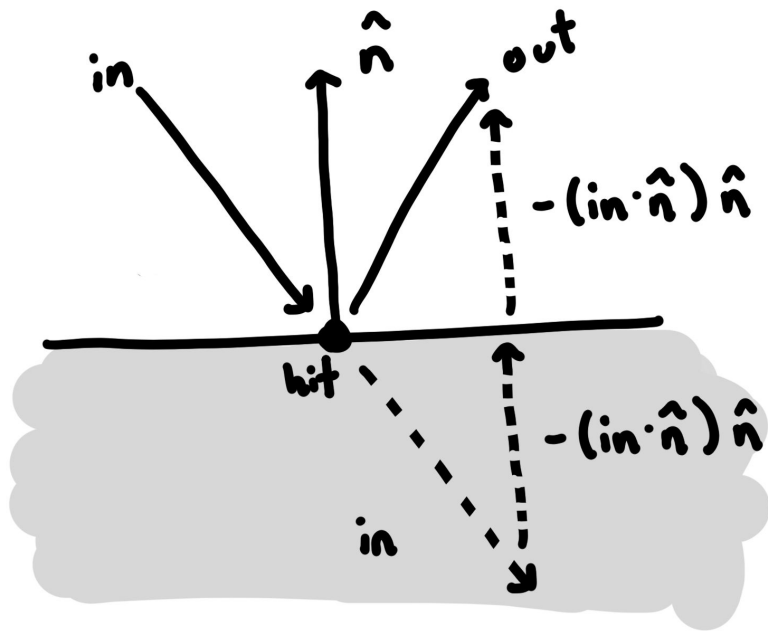
# DirectionalLight.cpp direction

- Input:
  - **q**: query point in space
- Output:
  - **d**: vector direction pointing towards this light's location **p**; normalized
  - **max\_t**: maximum (parametric) extent of this point light (i.e. how far this light can reach); parametric relative to **d**
- Check .h file for what members are defined



NOTE: `#include<limits>` has `std::numeric_limits<T>::infinity()`

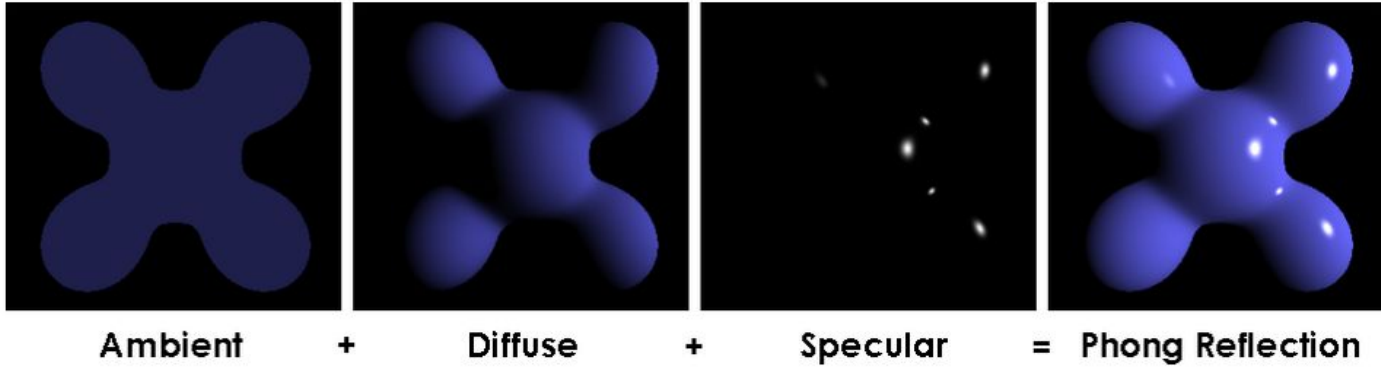
reflect.cpp



$$\mathbf{out} = \mathbf{in} - 2 * (\mathbf{in} \cdot \mathbf{n}) \mathbf{n}$$

NOTE: do not normalize – we want **out** to be the same length as **in**

blinn\_phong\_shading.cpp

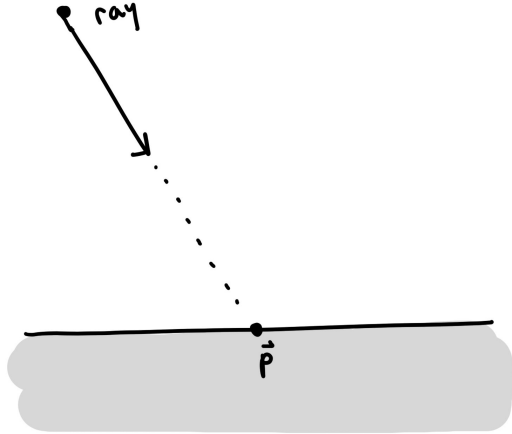




# blinn\_phong\_shading.cpp

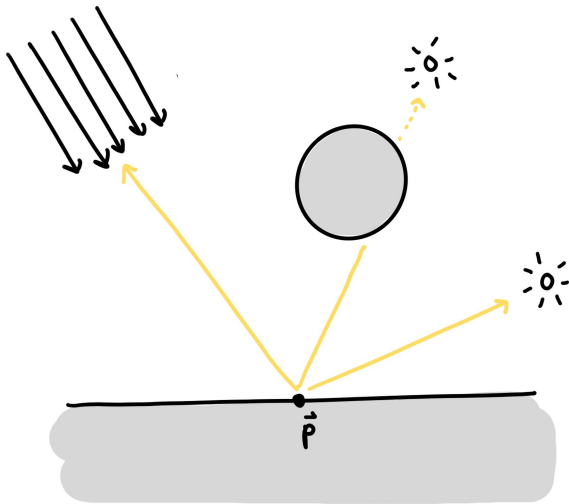
First, find intersection point  $\mathbf{p}$

$$\mathbf{p} = \text{ray.origin} + t * \text{ray.direction}$$



# blinn\_phong\_shading.cpp

Now, we want to sum the contributions from each light

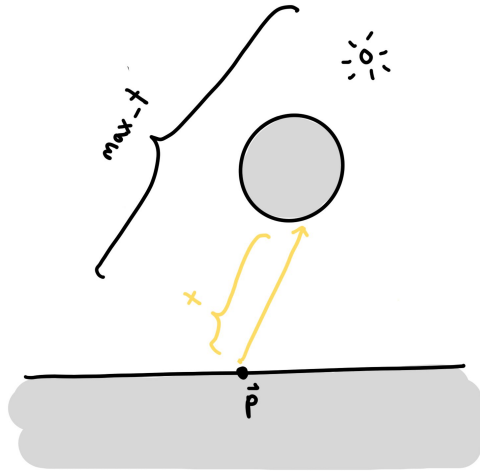


For each light

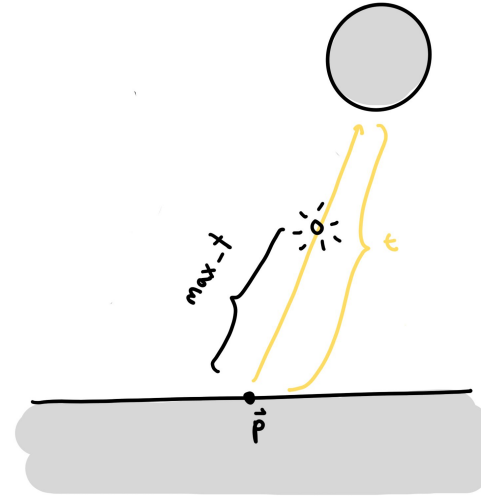
- check if we are in a shadow
  - ◆ construct `shadow_ray` from  $\vec{p}$  pointing towards the light
  - ◆ if the `shadow_ray` hits another object before it reaches the light (i.e. use `max_t`), then we ignore that light

# blinn\_phong\_shading.cpp

E.g. use `first_hit` to find  $t$



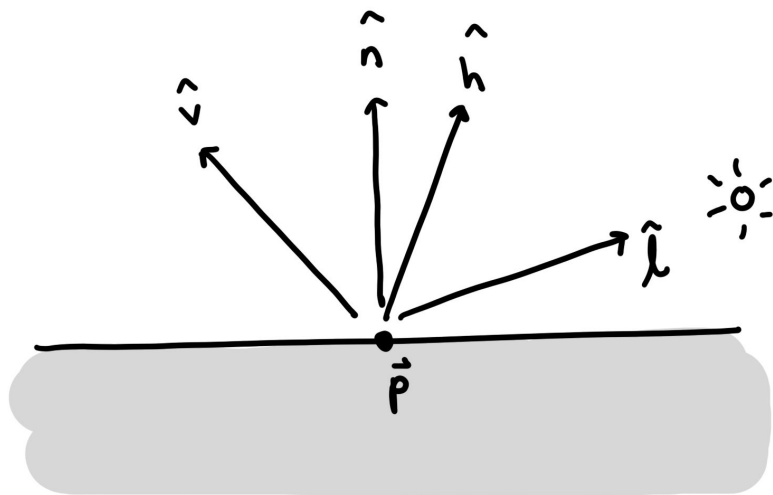
ignore



add

# blinn\_phong\_shading.cpp

If the point  $\mathbf{p}$  is lit:



$\mathbf{v}$ : view vector

- (i.e. **ray.direction** normalized)
- NOTE: **ray.direction** points towards  $\mathbf{p}$ , whereas Blinn-Phong is usually written with  $\mathbf{v}$  pointing away

$\mathbf{l}$ : light vector

- (i.e., same as **shadow\_ray.direction** normalized)

$\mathbf{h}$ : halfway vector

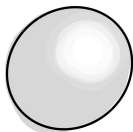
- $= (\mathbf{l} + \mathbf{v}) / \|\mathbf{l} + \mathbf{v}\|$
- NOTE: normalized

# blinn\_phong\_shading.cpp

$L += (\text{light color}) * \mathbf{k_s} * \max(0., \mathbf{n} \cdot \mathbf{h})^p$

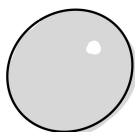
## specular light

- Phong exponent  $p$  controls object shininess



$p = 1$

vs

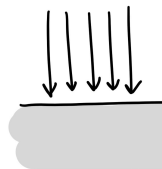


$p = 1000$

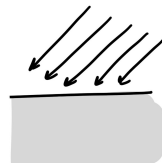
$L += (\text{light color}) * \mathbf{k_d} * \max(0., \mathbf{n} \cdot \mathbf{l})$

## diffuse light

- depends on angle between  $\mathbf{n}$  and  $\mathbf{l}$



is brighter than



# blinn\_phong\_shading.cpp

NOTE: include

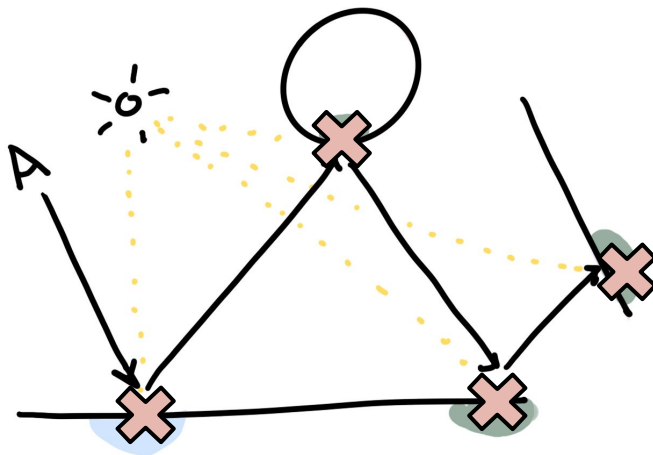
**L += ka \* ambient\_factor**

ambient light

- use 0.1 (see .h file)
- include only once (not for every light)
- represents the base color of the material if entirely in shadow

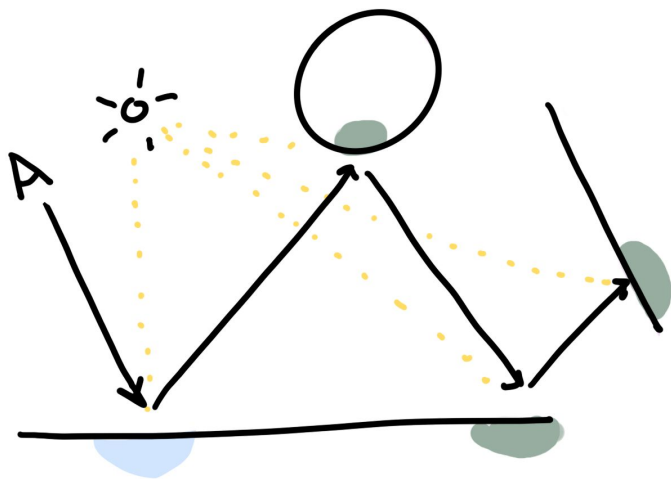
# raycolor.cpp

We want to implement recursive reflections



and sum the contributions of light color at each point of intersection

# raycolor.cpp



use `first_hit` to look for intersections, and if intersection is found:

- add object's color contribution using `blinn_phong_shading`
- make `mirror_ray`
- recurse
  - ◆ i.e. call `ray_color` then add  $\text{km} * (\text{returned recursion color})$  (where **km** is the material's mirror coefficient)

**NOTE:** only recurse up to a fixed recursion depth

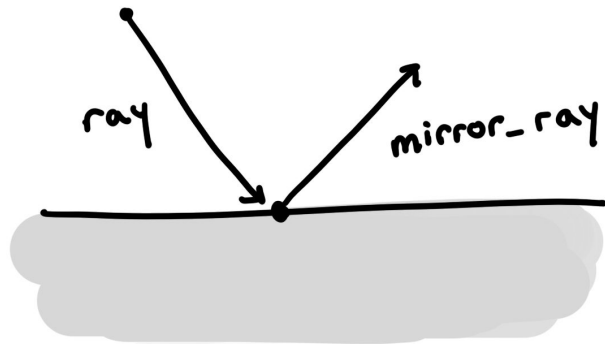


## raycolor.cpp

To construct `mirror_ray`:

**`mirror_ray.origin = ray.origin + t * ray.direction`**

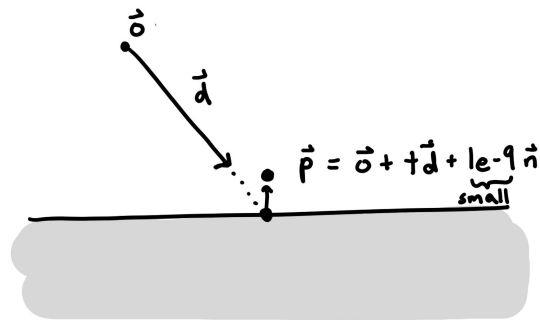
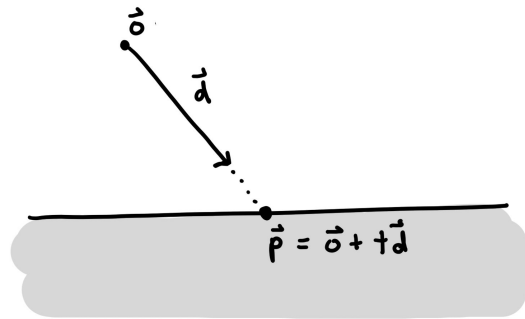
**`mirror_ray.direction = reflect()`**



## NOTE:

For **mirror\_ray.origin** and **shadow\_ray.origin**, might add a fudge factor since floating point rounding can cause the origins to be below the object's surface.

I.e. just add a very small surface normal to guarantee the origin is outside the surface



# creative.json

Can use any of the Object types (spheres, planes, triangles, triangle soups) implemented in ray casting.

See data/ for examples of .json scenes. (data/ also has some meshes available to use.)

